

Unroll List

–Class Tweaks

```
void displayList() const {...}
```

```
void searchElement(int val) const {...}
```

*Read – Only Guarantee: Placing **const** at the end of a method signature tells the compiler,*

"This function is only allowed to look at data, not alter it."

```
// Function to destroy the entire list and free all memory
void destroyList(bool printMessage = true) {
    if (head == nullptr) {
        if (printMessage) cout << "The list is already
empty.\n";
        return;
    }

    Node* temp = head;
    while (temp != nullptr) {
        Node* nextNode = temp->next;
        delete temp; // Use delete instead of free()
        temp = nextNode;
    }

    head = nullptr;
    if (printMessage) cout << "The entire list has been
destroyed and memory freed.\n";
}
```

We know delete keyword has two functions:

```
delete temp;
|
|-- 1. Call Destructor (~Node())    -> Cleans up internal resources (runs the default compiler destructor since our struct only holds basic data types)
|
|-- 2. Free Memory (operator delete) -> Releases the Node's own memory back to the Heap .
```

Note: `~Node()` this destructor manually not provided in the Node struct, C++ compiler automatically generates an implicit default destructor behind the scenes.

When the program executes `delete temp;`, it still calls `~Node()`, but it calls this invisible, compiler – generated version.

Why it is not in the code

We only need to manually write a destructor if an object manages its own complex resources (like opening a file, or using the new keyword to create internal variables). Our Node struct only contains primitive data types: an integer, an integer array, and a raw pointer. The compiler already knows exactly how to clean up primitive data types, so it handles it automatically.

```
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    Node* next;

    // Constructor
    Node(int val) {
        numElements = 1;
```

```

        elements[0] = val;
        next = nullptr;
    }

    // Explicit Destructor
    ~Node() {
        // Left empty because primitive types (int, int[], pointer)
        // are automatically destroyed by the compiler.
    }
};

```

Even though it is invisible in our current file, C++ guarantees that a destructor always exists and is always executed when we use the delete keyword.

And

```

~UnrolledLinkedList() {
    if (head != nullptr) {
        destroyList(false); // Suppress output during automatic
destruction
    }
}

```

***~UnrolledLinkedList()** is the destructor for the entire class manager. It only runs when the overarching list object (created in **main()**) finishes its lifecycle and is destroyed.*

*When we define **UnrolledLinkedList list;** in our **main()** function, the class object itself lives on the computer's temporary Stack memory, while all the individual nodes it creates are dynamically allocated on the Heap.*

When the program ends, as the object simply goes out of scope, the compiler automatically triggers `~UnrolledLinkedList()`. The destructor acts as the bridge: it uses our custom `destroyList(false)` loop to actively sweep the Heap and `delete` every single node, ensuring no memory leaks remain before the class object itself is naturally erased from the Stack.

```
~UnrolledLinkedList() {  
    if (head != nullptr) {  
        destroyList(false); // Suppress output during automatic  
destruction  
    }  
}
```

And,

```
// Function to destroy the entire list and free all memory  
void destroyList(bool printMessage = true) {  
    if (head == nullptr) {  
        if (printMessage) cout << "The list is already  
empty.\n";  
        return;  
    }  
  
    Node* temp = head;  
    while (temp != nullptr) {  
        Node* nextNode = temp->next;  
        delete temp; // Use delete instead of free()  
        temp = nextNode;  
    }  
  
    head = nullptr;  
    if (printMessage) cout << "The entire list has been  
destroyed and memory freed.\n";  
}
```

As it pass ***destroyList(bool printMessage = true)***, as it passes ***false*** ,
in the argument, then:

The single if statements :

```
if (printMessage) cout << "The list is already  
empty.\n";
```

And,

```
if (printMessage) cout << "The entire list has been  
destroyed and memory freed.\n";  
}
```

becomes false and body of the if statements never executes.

These two if bodies only executes then:

```
destroyList(true);
```
